Deterministic Optimization

Discrete Optimization: Branch-and-Bound

Shabbir Ahmed
Anderson-Interface Chair and Professor
School of Industrial and Systems Engineering

The Branch-and-Bound Algorithm

The Branch-and-Bound Algorithm

Learning Objectives

- Inspect the branch-and-bound algorithm for solving integer programs

LP Relaxation

- We would like to solve the integer program:

$$\begin{aligned} v_{IP} = \min \quad & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} \geq \mathbf{b} \\ & \mathbf{x} \in \mathbb{R}^{n-p} \times \mathbb{Z}^p \end{aligned}$$

- First solve the LP relaxation.
- If the LP relaxation is infeasible then the IP is infeasible. We can stop.
- If the LP relaxation is unbounded then, Assuming rational data, the IP is either infeasible or unbounded. We can stop.

LP Relaxation Solution

- Suppose the LP relaxation has an optimal solution.
  Let v_{LP} denote its optimal value and $\mathbf{x}^{LP}$ be an optimal solution.
- If it happens that the LP relaxation solution is integral, that is $\mathbf{x}^{LP} \in \mathbb{R}^{n-p} \times \mathbb{Z}^p$, then we know that $\mathbf{x}^{LP}$ is an optimal solution to the IP and $v^{IP} = v^{LP}$. So we can stop.
- Suppose that the LP relaxation solution is not integral, i.e. there is a component j of the solution vector that is supposed to integral but now has a fractional value, i.e. $x_j^{LP} \notin \mathbb{Z}$. We also know that $v^{LP} \leq v^{IP}$.

Branching

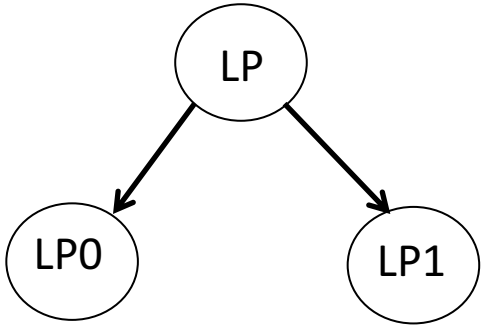
- Create two new LPs:

$$(P0): \quad v_0^{LP} = \min \ \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \ \mathbf{A}\mathbf{x} \geq \mathbf{b}, \ x_j \leq \lfloor x_j^{LP} \rfloor$$

$$(P1): \quad v_1^{LP} = \min \ \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \ \mathbf{A}\mathbf{x} \geq \mathbf{b}, \ x_j \geq \lceil x_j^{LP} \rceil$$

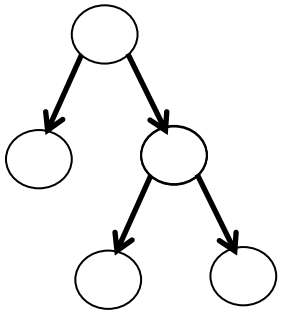
- The optimal solution to the IP (if it exists) must be contained in the feasible region of the one of the two LPs.
- Note that $v^{IP} \geq \min\{v_0^{LP}, v_1^{LP}\}$
- Solve these LPs and proceed recursively.

Bounding



- If any of the LPs return an integer solution then we have feasible solution to the IP. If the objective value of this solution is better (smaller) then any we have found so far, then we keep it as an "incumbent" solution and maintain the objective as the best upper bound UB, i.e. $v^{IP} \leq U$.
- If an LP is infeasible, then we need not explore it any further.
- If an LP has an objective value that is worse that UB then we need not explore this any further.

Branch-and-Bound Tree



- Any LP that has an integer optimal solution need not be explored further.
- The LPs that need not be explored further are deemed to be “fathomed” or “pruned”
- An LP that has not been fathomed need to be branched creating two new LPs
- This process is arranged into a (inverted) tree where each LP is called a node, and the initial LP relaxation is the “root” node.

Lower Bound

- At any point in time this tree has some nodes (LPs) that are pruned and has some nodes that need to further explored.
- At any point in the recursive process, the minimum of the objective values of the LPs that still need to be explored provide a lower bound LB.
- We can stop the search if the LB and UB are close together and return the incumbent solution as a feasible solution with an optimality gap of at most $UB - LB$.

Branch and Bound

- The branch-and-bound scheme simply relies on the solutions of LPs obtained by adding constraints to the root LP relaxation
- The same idea can be extended to nonlinear IPs as long as we can solve the relaxations exactly.
- For linear integer programs, the basic algorithm is enhanced by many heuristics to obtain a feasible integer solution from the node LPs and by adding valid inequalities to improve the formulation of the node LPs.

Summary

- We discussed the branch-and-bound algorithm which is the backbone of IP solvers.
- We will observe a small example of branch-and-bound in the next lesson